

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Computação Gráfica

Ano Letivo de 2025/2026

Fase 2 - Grupos e Transformações

Carlos Araújo
A106845

João Teixeira
A106836

Simão Mendes
A106928

Vitor Senra
A106921

March 16, 2026

CG

Índice

1. Introdução	1
2. Grupos	2
2.1. Estrutura Hierárquica de Grupos	2
2.1.1. Estrutura e Composição	2
2.1.2. Carregamento Recursivo (XML)	2
2.1.3. Algoritmo de Renderização (draw)	2
3. Transformações	4
3.1. Transformações Geométricas	4
3.1.1. Estrutura das Transformações	4
3.1.2. Implementação Técnica (<i>Apply</i>)	4
3.1.3. Ordem e Acumulação	4
4. Resultados e Validação	6
4.1. Testes da Equipa Docente	6
4.2. Modelo do Sistema Solar	7
4.2.1. Resumo dos Elementos Principais	8
5. Extras	10
5.1. Formato <i>Wavefront</i> (Adaptação do ficheiro .3d)	10
5.2. <i>Torus</i>	10
5.3. Câmara Orbital	12
5.4. Câmara Livre (<i>Free Camera</i>)	13
6. Conclusão	14
6.1. Dificuldades Encontradas	14
7. Referências Bibliográficas	15

1. Introdução

Nesta segunda fase do projeto de Computação Gráfica, o objetivo principal focou-se na **expansão do motor 3D**, desenvolvido na fase anterior, para suportar **transformações geométricas hierárquicas**.

Para alcançar este propósito, a equipa reestruturou profundamente a arquitetura do motor através da adoção de **grupos hierárquicos**. Esta estrutura de dados em árvore permite organizar modelos e operações matemáticas (translações, rotações e escalas) em blocos aninhados. Desta forma, as transformações aplicadas a um grupo propagam-se automaticamente a todos os seus subgrupos e elementos descendentes, o que facilita a construção de cenários complexos.

Adicionalmente, o grupo melhorou o sistema de visualização com a **criação de duas câmaras interativas** (uma orbital e uma livre) e expandiu o gerador de figuras com uma **nova forma paramétrica**, o *Torus*. Para validar e demonstrar todas estas novas funcionalidades, concebemos um **modelo estático e realista do Sistema Solar**, repleto de planetas, luas e um cinturão de asteroides denso.

2. Grupos

2.1. Estrutura Hierárquica de Grupos

A organização da cena baseia-se na classe `Group`, que funciona como o nó fundamental de uma árvore de cena (*Scene Graph*). A implementação utiliza uma estratégia de **composição**, onde cada grupo atua como um contentor autónomo de estados, modelos e subgrupos.

2.1.1. Estrutura e Composição

Cada instância da classe `Group` gere três listas fundamentais utilizando o contentor `std::vector` pela sua eficiência e facilidade de manipulação:

- **Transformações (transforms):** Um vetor de objetos `Transform` que define as operações geométricas aplicadas globalmente ao grupo.
- **Modelos (models):** Um vetor de objetos `Model` que armazena a geometria (vértices) a ser renderizada neste nível hierárquico.
- **Subgrupos (children):** Um vetor de objetos `Group`, permitindo a definição de hierarquias complexas de forma recursiva.

2.1.2. Carregamento Recursivo (XML)

O carregamento da cena é realizado pelo método `loadFromXML` (já utilizado na primeira fase), que processa o ficheiro de configuração de forma descendente:

1. **Transformações:** O motor identifica elementos como `<translate>`, `<rotate>` ou `<scale>`, extrai os seus atributos (`x`, `y`, `z`, `angle`) e armazena-os por ordem de declaração no vetor de transformações.
2. **Modelos:** Dentro do bloco `<models>`, cada tag `<model>` tem o seu ficheiro `.3d` correspondente carregado para uma instância da classe `Model`.
3. **Recursividade:** Sempre que uma tag `<group>` é encontrada dentro de outra, o sistema instancia um novo grupo e invoca novamente o método `loadFromXML`. Isto garante que a árvore de cena em memória reflita exatamente a estrutura aninhada do ficheiro XML.

2.1.3. Algoritmo de Renderização (draw)

A renderização respeita a herança de transformações através da manipulação da pilha de matrizes do OpenGL:

- **Isolamento:** O uso de `glPushMatrix()` no início do método garante que as transformações aplicadas num grupo não afetem elementos fora da sua ramificação hierárquica.

- **Acumulação:** As transformações do grupo são aplicadas sequencialmente. Devido à natureza da pilha do OpenGL, estas acumulam-se com as transformações já herdadas dos grupos “pais”.
- **Desenho e Propagação:** Após desenhar os modelos locais, o grupo chama o método `draw()` de todos os seus filhos. Este processo garante que um objeto filho (como uma Lua) herde automaticamente a posição e rotação do seu pai (como a Terra).
- **Restauração:** O comando `glPopMatrix()` é usado no fim do processamento de um grupo, garantindo que as mudanças locais não interfiram com os grupos seguintes.

Esta arquitetura permite criar sistemas complexos, como o Sistema Solar, onde a dependência entre astros é gerida de forma transparente pela estrutura de dados.

3. Transformações

3.1. Transformações Geométricas

O motor de renderização utiliza a estrutura `Transform` para gerir todas as operações geométricas aplicadas aos objetos da cena. Esta implementação centraliza a lógica de manipulação espacial, garantindo que as transformações sejam aplicadas de forma eficiente e previsível.

3.1.1. Estrutura das Transformações

Cada transformação é instanciada com um tipo específico, definido através de um enumerador (`enum class TransformType`), com as seguintes possibilidades de valores:

- **TRANSLATE**: Define o deslocamento do sistema de coordenadas nos eixos x , y e z .
- **ROTATE**: Define uma rotação com um determinado *angle* (ângulo) em torno de um eixo definido pelo vetor (x, y, z) .
- **SCALE**: Define o fator de redimensionamento aplicado a cada um dos eixos cartesianos.

Para além do tipo, a estrutura armazena os valores flutuantes x , y , z que indicam a magnitude ou direção da transformação e o valor *angle*, utilizado exclusivamente no caso de uma rotação.

3.1.2. Implementação Técnica (*Apply*)

A aplicação prática destas transformações é realizada pelo método `apply()`, que interage diretamente com as funções de estado do OpenGL através de uma estrutura de decisão baseada no tipo da transformação:

- No caso de uma translação, é invocada a função `glTranslatef(x, y, z)`.
- Para rotações, utiliza-se `glRotatef(angle, x, y, z)`.
- Para escalas, aplica-se `glScalef(x, y, z)`.

3.1.3. Ordem e Acumulação

É nesta parte que entra a importância da classe `Group`. Como cada grupo possui a sua própria lista de transformações (`std::vector<Transform>`), o processo de desenho de um grupo inicia-se sempre pela iteração sobre esta lista, chamando o método `apply()` para cada elemento antes de proceder ao desenho dos modelos ou dos subgrupos.

Esta organização garante que a ordem de declaração no ficheiro XML seja estritamente respeitada. Uma vez que o OpenGL utiliza uma pilha de matrizes, as transformações são acumuladas sequencialmente, permitindo que subgrupos herdem as transformações dos seus antecessores de forma automática e organizada.

4. Resultados e Validação

4.1. Testes da Equipe Docente

Dando continuidade à validação do projeto, esta secção apresenta os resultados obtidos na **Fase 2**. Mantendo a metodologia anterior, cada teste foi processado pelo *Generator* e renderizado na *Engine*.

Confrontamos os resultados obtidos com os **frames** de referência, sendo a fidelidade visual entre ambos (lado esquerdo vs. lado direito) a prova da implementação bem-sucedida das primitivas específicas desta fase.

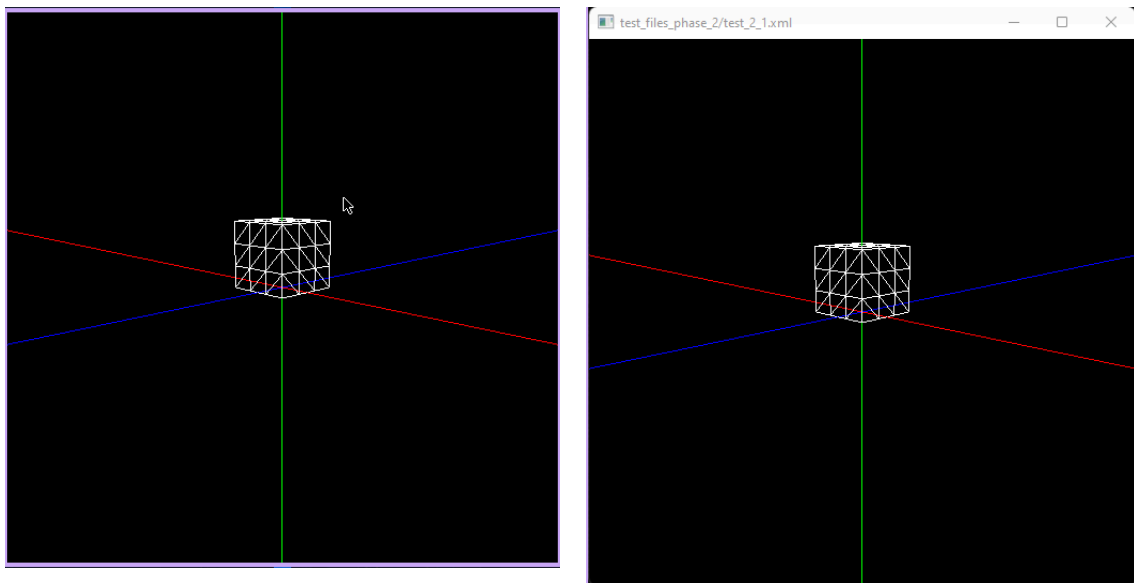


Figura 1: Comparação visual para o teste 2_1.

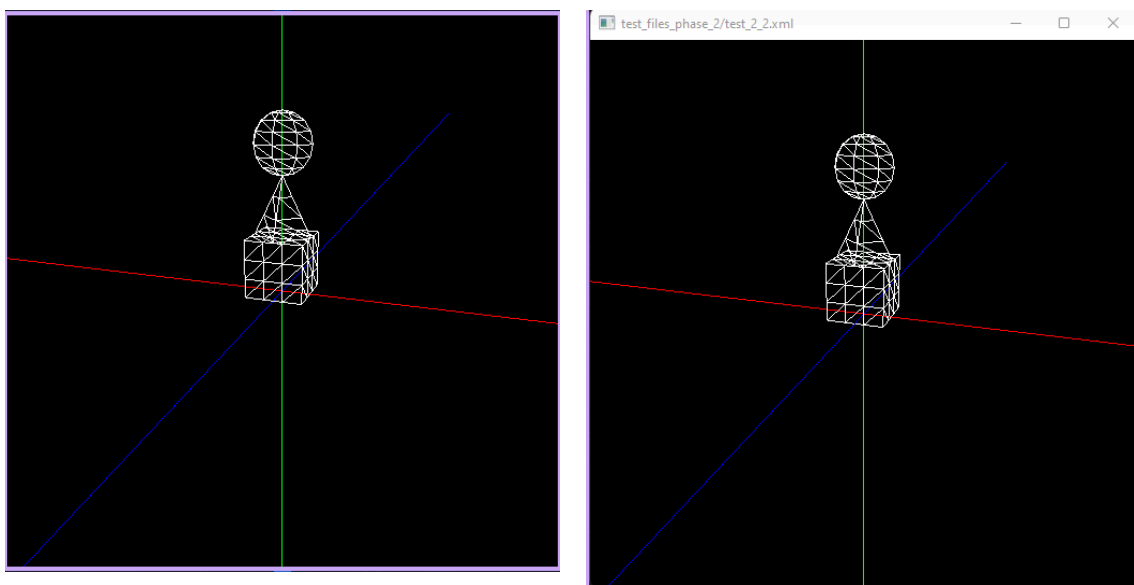


Figura 2: Comparação visual para o teste 2_2.

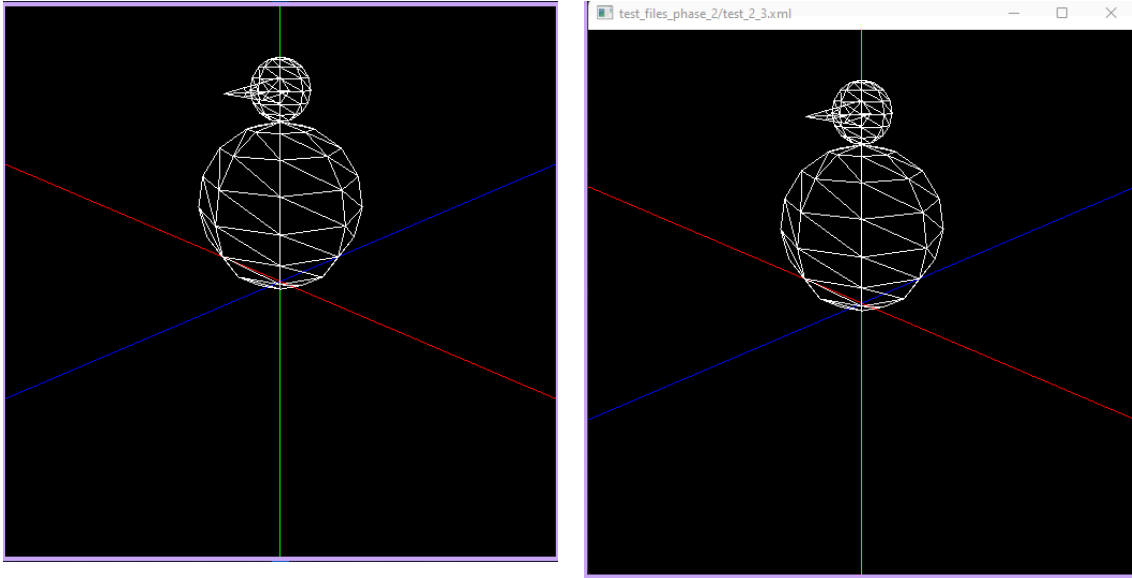


Figura 3: Comparação visual para o teste 2_3.

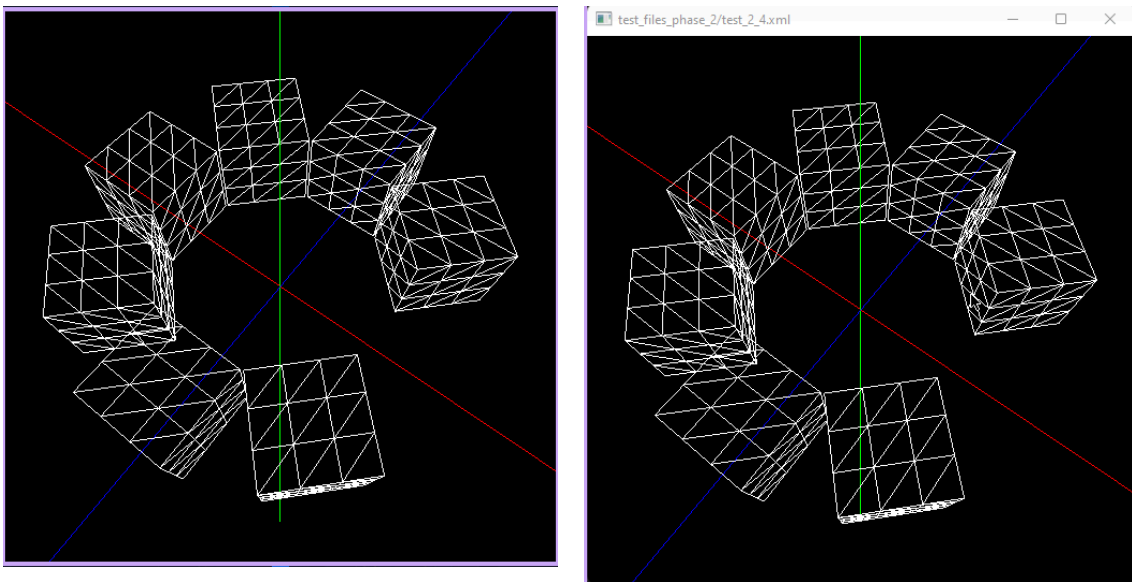


Figura 4: Comparação visual para o teste 2_4.

4.2. Modelo do Sistema Solar

A representação do sistema solar foi implementada recorrendo a uma hierarquia de grafos de cena, onde a estrutura de agrupamentos (<group>) permite a aplicação sucessiva de transformações geométricas, garantindo a correta relação entre os astros e as suas órbitas.

A construção do modelo baseia-se nos seguintes princípios técnicos:

- **Hierarquia e Encapsulamento:** Cada planeta e as suas respectivas luas são definidos dentro de blocos group isolados, o que assegura que as transformações locais (como translação e escala) não afetem o restante sistema.

- **Representação de Corpos Celestes:** A maioria dos objetos utiliza a primitiva `sphere.3d`, com exceção do sistema de anéis de Saturno, que implementa o modelo `torus.3d`.
- **Geometria dos Anéis:** Para simular o aspeto achatado dos anéis de Saturno, foi aplicada uma transformação de escala específica de $(1.4, 0.1, 1.4)$, convertendo a primitiva toroidal num disco fino.
- **Distribuição da Cintura de Asteroides:** A implementação da cintura de asteroides consiste num ciclo de 72 objetos, distribuídos angularmente através de rotações incrementais de 5° ao longo do eixo y , com variações em y e escala para conferir dispersão natural.

4.2.1. Resumo dos Elementos Principais

Objeto	Modelo	Escala	Distância ao Centro
Sol	<code>sphere.3d</code>	12.0	0
Mercúrio	<code>sphere.3d</code>	0.4	18
Vénus	<code>sphere.3d</code>	0.9	26
Terra	<code>sphere.3d</code>	1.0	35
Lua	<code>sphere.3d</code>	0.25	+2.0 (da Terra)
Marte	<code>sphere.3d</code>	0.5	45
Luas (x2)	<code>sphere.3d</code>	0.08 a 0.1	+1.2 a 1.8 (de Marte)
Cinturão de Asteroides	<code>sphere.3d</code>	0.3 a 0.8	58 a 62.2
Júpiter	<code>sphere.3d</code>	5.0	80
Luas (x4)	<code>sphere.3d</code>	0.25 a 0.4	+6.0 a 11.5 (de Júpiter)
Saturno	<code>sphere.3d</code>	4.2	110
Anel	<code>torus.3d</code>	$(1.4, 0.1, 1.4)$	Centro de Saturno
Titã	<code>sphere.3d</code>	0.4	+12.0 (de Saturno)
Úrano	<code>sphere.3d</code>	2.5	140
Neptuno	<code>sphere.3d</code>	2.4	170

Esta estrutura demonstra a eficácia do uso de matrizes de transformação encadeadas, permitindo que a movimentação de um planeta pai arraste automaticamente os seus satélites naturais.

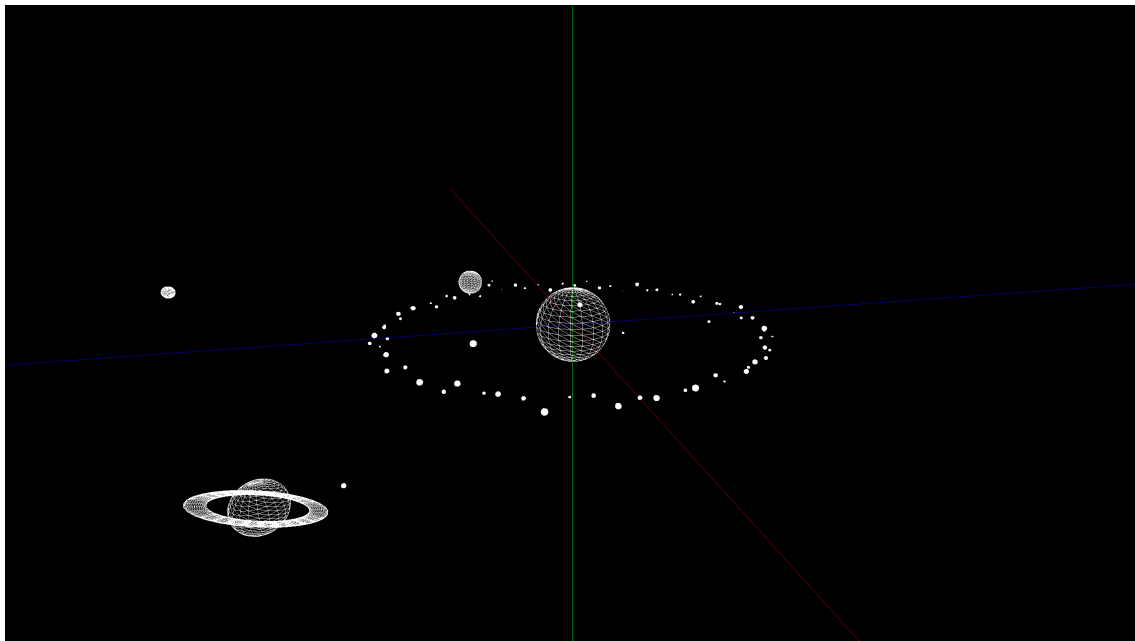


Figura 5: Modelo do sistema solar desenvolvido

5. Extras

Nesta secção, documentam-se as funcionalidades adicionais implementadas no projeto, que complementam os requisitos base da segunda fase e expandem as capacidades do motor 3D.

5.1. Formato *Wavefront* (Adaptação do ficheiro .3d)

De forma a otimizar a representação de modelos e alinhar o projeto com as convenções da indústria, o formato de ficheiro proprietário .3d utilizado pelo nosso motor foi reestruturado para seguir nativamente a norma e a arquitetura do formato genérico *Wavefront* (.obj).

Esta alteração estrutural dita que a informação do modelo deixa de ser apenas uma lista de triângulos e passa a estar indexada e organizada por blocos, identificados através de prefixos específicos no início de cada linha:

- **Vértices (v):** Define as coordenadas espaciais geométricas (x, y, z) de cada ponto único do modelo no espaço 3D.
- **Faces (f):** Constrói os polígonos da malha (triângulos) utilizando índices numéricos que apontam para os vértices (v) previamente declarados. A definição faz-se agrupando os índices de cada ponto que compõe a face, tipicamente no formato `f v1 v2 v3`.

A transição para este formato indexado inspirado no *Wavefront* elimina a redundância de dados (evitando a repetição das coordenadas de vértices partilhados por vários triângulos) e prepara a base arquitetural do motor de forma muito mais robusta para suportar as exigências gráficas das fases subsequentes.

5.2. *Torus*

A figura geométrica do *Torus* foi adicionada ao Gerador. A sua malha é construída dividindo a forma em fatias (slices) e pilhas (stacks), o que gera uma grelha de faces poligonais.

A modelação matemática baseia-se em geometria toroidal. Considerando as convenções padrão para a criação desta figura, definimos dois raios fundamentais:

- r_0 (**outerRadius**): O raio principal, medido desde o centro absoluto da figura até ao meio do tubo (anel do *Torus*).
- r_1 (**innerRadius**): O raio da secção transversal, ou seja, o raio do próprio tubo.

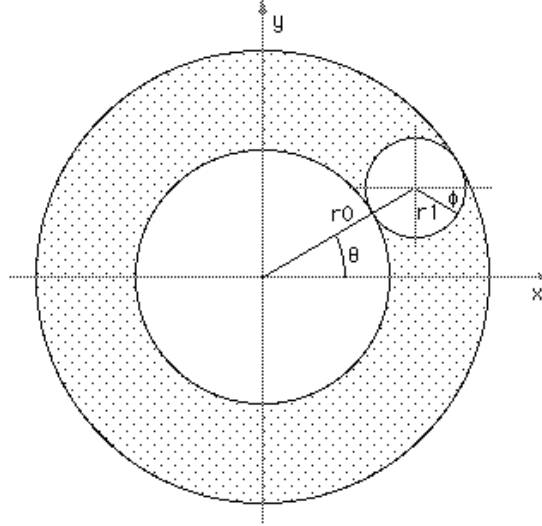


Figura 6: Diagrama explicativo dos raios (r_0 e r_1) e dos ângulos (θ e φ) na geometria toroidal.

As equações paramétricas utilizadas para calcular cada ponto (x, y, z) na superfície do Torus, onde os ângulos θ e φ variam entre 0 e 2π , são definidas da seguinte forma:

$$x = (r_0 + r_1 \cdot \cos(\varphi)) \cdot \cos(\theta)$$

$$y = r_1 \cdot \sin(\varphi)$$

$$z = (r_0 + r_1 \cdot \cos(\varphi)) \cdot \sin(\theta)$$

Geração da Malha (*Facets*): Para criar uma aproximação poligonal do *Torus*, o nosso algoritmo calcula os vértices iterando sobre os ângulos com incrementos $d\theta$ e $d\varphi$ (valores que derivam diretamente do número de *slices* e *stacks*). Cada face quadrangular é definida por quatro vértices no espaço paramétrico: (θ, φ) , $(\theta + d\theta, \varphi)$, $(\theta + d\theta, \varphi + d\varphi)$ e $(\theta, \varphi + d\varphi)$. O Gerador subdivide imediatamente este quadrilátero em dois triângulos para permitir o correto processamento pelo *pipeline* do *OpenGL*.

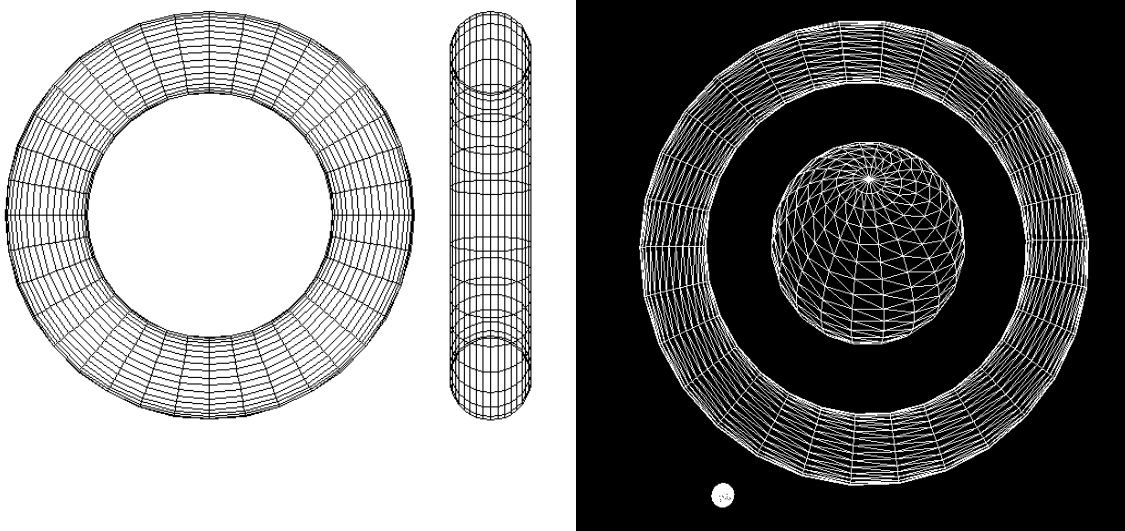


Figura 7: Comparação entre a malha teórica facetada de um *Torus* (à esquerda) e a malha efetivamente gerada pelo nosso motor 3D, aplicada aos anéis de Saturno na demonstração (à direita).

5.3. Câmara Orbital

A nova Câmara Orbital permite uma navegação fluida em torno de um ponto de interesse na cena, recorrendo a interações com o rato e o teclado. Em vez de calcular rotações complexas utilizando álgebra linear, a câmara apoia-se nativamente num modelo de coordenadas esféricas.

A cada *frame*, os dados esféricos são convertidos em coordenadas cartesianas para determinar o vetor posição da câmara através das seguintes equações matemáticas:

$$x = r \cdot \cos(\beta) \cdot \sin(\alpha)$$

$$y = r \cdot \sin(\beta)$$

$$z = r \cdot \cos(\beta) \cdot \cos(\alpha)$$

Os parâmetros operam de forma independente:

- **r (radius)** define a distância entre a câmara e o ponto de foco. A aproximação e o afastamento da visão (zoom) são efetuados manipulando este valor através das teclas ‘W’ e ‘S’, garantindo sempre que não é inferior a um limite mínimo estabelecido.
- **α (Alpha) e β (Beta)** representam, respetivamente, a rotação horizontal e a elevação vertical da câmara em torno da origem.

Durante a fase de implementação, verificou-se que a câmara sofria de inversões de eixo indesejadas sempre que ultrapassava os polos de observação (o “*gimbal lock*” comum nestes sistemas). A equipa resolveu este problema estabelecendo limites matemáticos restritos: o ângulo de elevação β foi delimitado ao intervalo restrito entre -1.5 e 1.5 radianos, bloqueando a vista suavemente antes da inversão polar.

5.4. Câmara Livre (*Free Camera*)

Para além da câmara orbital, foi também implementada uma Câmara Livre (frequentemente designada por câmara **First-Person** ou de exploração), que permite ao utilizador navegar pelo ambiente 3D sem estar restrito a um ponto de foco central. Esta funcionalidade é especialmente útil para inspecionar cenas complexas em detalhe.

A matemática que sustenta esta câmara baseia-se no cálculo contínuo de um vetor de direção (**forward vector**) a partir de dois ângulos de orientação capturados pelo movimento do rato: o **Yaw** (rotação horizontal do pescoço) e o **Pitch** (inclinação vertical da cabeça). A cada *frame*, o motor recalcula o vetor para onde a câmara está a olhar recorrendo à seguinte base trigonométrica:

$$d_x = \cos(\text{pitch}) \cdot \sin(\text{yaw})$$

$$d_y = \sin(\text{pitch})$$

$$d_z = \cos(\text{pitch}) \cdot \cos(\text{yaw})$$

Para permitir o movimento lateral do observador (**strafing**), o motor precisa de conhecer o eixo horizontal da câmara. Esse vetor lateral (**right vector**) é calculado através do produto vetorial (**cross product**) entre o vetor de direção da câmara e o vetor vertical global da cena (o **Up Vector**, tipicamente definido como $(0, 1, 0)$).

O movimento físico no espaço é depois executado mapeando as teclas de navegação padrão ('W', 'A', 'S', 'D'). Estas teclas atualizam as coordenadas reais (x, y, z) da posição da câmara transladando o ponto de visão ao longo do vetor de direção (frente/trás) ou do vetor lateral (esquerda/direita), multiplicados por um fator de velocidade.

À semelhança da solução encontrada para a câmara orbital, o ângulo de **pitch** foi matematicamente restringido ao intervalo $[-1.5, 1.5]$ radianos (aproximadamente $[-89^\circ, 89^\circ]$). Esta limitação é crucial para evitar o problema de **gimbal lock** e impedir que a câmara sofra uma rotação invertida indesejada.

6. Conclusão

A conclusão desta segunda fase atesta o sucesso da implementação dos **grupos hierárquicos** e da nova arquitetura do motor 3D. Através da gestão rigorosa da pilha de matrizes do OpenGL, com o uso das funções `glPushMatrix()` e `glPopMatrix()`, o programa consegue agora processar ficheiros XML com múltiplos níveis de profundidade recursiva, de forma a isolar corretamente as transformações de cada nó.

O modelo do **Sistema Solar** validou, na prática, a eficácia destas alterações matemáticas. A colocação de luas na órbita de planetas, a inclinação dos anéis de Saturno (gerados através da nova figura *Torus*) e a criação de um cinturão de asteroides provaram que a hierarquia de transformações funciona de forma imaculada e sem distorções visuais.

Por fim, a inclusão das **câmaras interativas** revelou-se uma melhoria crucial para o projeto. A câmara orbital em coordenadas esféricas facilitou a inspeção minuciosa das escalas e posições focadas nos astros, enquanto a nova câmara livre proporcionou uma navegação total e imersiva pelo cenário. O motor 3D encontra-se, assim, robusto e perfeitamente preparado para a terceira fase, que introduzirá movimento contínuo e animação através de curvas.

6.1. Dificuldades Encontradas

Durante o desenvolvimento desta fase, a equipa deparou-se com alguns desafios técnicos significativos:

- **Gestão da Pilha de Matrizes:** O primeiro grande obstáculo residiu na aplicação precisa das transformações. Um pequeno lapso na ordem das invocações das matrizes resultava na propagação incorreta de translações para os nós irmãos (por exemplo, a órbita da Terra afetar a órbita de Marte). Foi necessário aplicar um rigor extremo no código para isolar o estado de cada subgrupo.
- **Leitura Recursiva de XML:** A adaptação do *parser* exigiu um desenho estrutural cuidadoso. O motor precisava de ler grupos dentro de grupos sem qualquer limite de profundidade e sem causar falhas de memória ou ciclos infinitos na instanciação dos objetos.
- **Matemática da Câmara Orbital:** A conversão contínua de coordenadas esféricas para cartesianas exigiu testes exaustivos. Para além disso, a câmara sofria de inversões indesejadas nos polos. A equipa resolveu este problema com sucesso através da imposição de limites matemáticos rígidos ao ângulo vertical da visão.

7. Referências Bibliográficas

- [1] D. Shreiner, M. Woo, J. Neider, e T. Davis, *OpenGL Programming Guide*, 8.º ed. Addison Wesley, 2013.
- [2] L. Thomason, «tinyxml2». [Em linha]. Disponível em: <https://github.com/leethomason/tinyxml2>
- [3] Library of Congress, «Wavefront OBJ File Format». [Em linha]. Disponível em: <https://www.loc.gov/preservation/digital/formats/fdd/fdd000508.shtml>
- [4] J. De Vries, «LearnOpenGL: Camera». [Em linha]. Disponível em: <https://learnopengl.com/Getting-started/Camera>
- [5] P. Bourke, «Toroidal Geometry». [Em linha]. Disponível em: <https://paulbourke.net/geometry/toroidal/>